

ASSIGNMENT 1: ACTIVITY RECOGNITION PROJECT

CSE 572: DATA MINING

A PROJECT REPORT

Submitted by

AKSHAY DESHNENI (ASU ID: 1218717838)

In partial fulfillment for the completion of the course CSE 572: Data Mining



ARIZONA STATE UNIVERSITY

February 9, 2020

Contents

1. Introduction
2. Project Phase I
3. Project Phase II
4. Project Phase III
5. Bibliography

Introduction

In this project, an attempt has been made to develop a computing system that can understand human activities. Data is provided for eating action mixed with other unknown activities. The data consists of 8 EMG sensors for a single user. The aim of the assignment is to identify the eating activities amongst the noise. The assignment is comprised of three phases. In the first phase we will do data cleaning, noise reduction, data organization and feature extraction. In the second phase, we will implement six existing feature extraction and statistical methods such as standard deviation, mean, min, max, root mean square (RMS) and fast fourier transform (FFT). The aim is to use these features to show clear distinction between the actions. In the third phase, we reduce the feature space and keep only those features which show maximum distance between the two classes (eating and non-eating). We will use principal component analysis (PCA) for this purpose. All file names mentioned in this project report here onwards are available in the project folder for your reference. The entire project was coded in Python using some of its libraries like numPy, pandas, scikit-Learn and matplotlib.

Project Phase I: Data Cleaning and Organization

The data provided is collected primarily using two sources:

- A) Myo Wristband Data: The subject was wearing the wristband and performed eating actions periodically interspersed with non-eating unknown actions. The wristband provides with i) accelerometer, ii) gyroscope, iii) orientation and iv) EMG sensors. But for the scope of this assignment we are going to only consider EMG sensors for a single user 16. The sampling rate of EMG sensors is 100 Hz.
- B) Ground Truth Data: Video recording of the person performing eating acting actions is used for establishing ground truth data. The ground truth data is provided in the form of frame numbers where an eating action starts and ends. The video data is taken at 30 frames per second.

We must first convert the starting and ending eating action frame numbers to sample numbers in the first step of this assignment. Also, in this assignment we assume that the start frame and sample #1 of the wristband are synchronized.

Ground Truth data for Eating Fork action is given in the file 1503600471339.txt for user

16. Path: /groundTruth/user16/fork

Ground Truth data for Eating Spoon action is given in the file 1503599958929.txt for user

16. Path: /groundTruth/user16/spoon

The first column of these files is the 'Start' frame number and the second column is the 'End' frame number.

Each row is an eating action. We ignore the third column from the scope of this assignment.

EMG data for Fork is given in the file '1503600471339_EMG.txt' for user 16.

Path: /MyoData/user16/fork

EMG data for Spoon is given in the file '1503599958929_EMG.txt' for user 16.

Path: /MyoData/user16/spoon

EMG sensor's sampling rate is 100 Hz and the video samples were recorded at 30 frames per second.

Therefore, in order to convert the frame-start and frame-end numbers to sample numbers we need to multiply each frame number by 100 and the divide by 30.

We wrote a python code for this process and it is given in the file 'GroundTruthCleaned.py'

Output of this program will give two matrices:

- i) 'groundTruthForkCleaned.csv' and
- ii) 'groundTruthSpoonCleaned.csv'

Path: /Phase1

After this we map the sample numbers with the sample numbers from the EMG data for Fork and Spoon to get the eating and non-eating samples.

The python code for this process is given in the file 'Eating.py'

We got four matrices at the end of this phase:

- i) 'FinalEatingFork.csv'
- ii) 'FinalEatingSpoon.csv'
- iii) 'FinalNotEatingFork.csv'
- iv) 'FinalNotEatingSpoon.csv'

Path: /Phase1

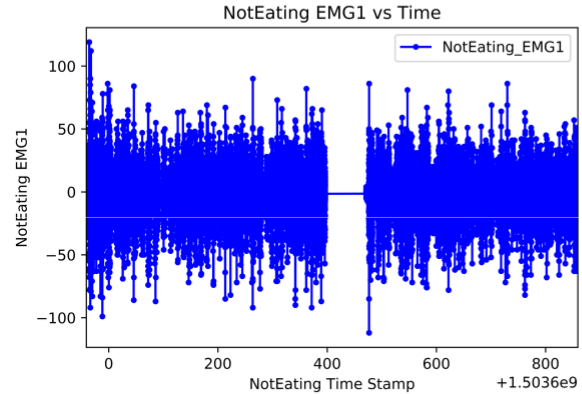
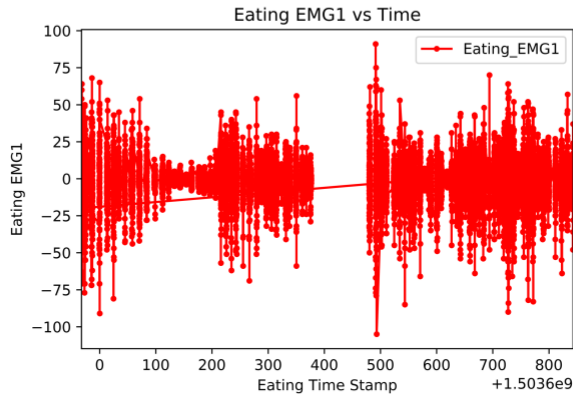
Project Phase II: Feature Extraction

This phase involves in the implementation of the feature extraction methods and generation of plots on the actions performed to visualize the features. From the last phase we have separated the EMG sensor data for Eating and Non-Eating actions based on the video frame numbers of the ground truth. We have got four matrices for Eating (Fork), Eating (Spoon), NotEating (Fork) and NotEating (Spoon). In order to visualize the raw data from the sensors we have plotted each sensor (EMG1 to EMG8) with respect to their timestamps using python's matplotlib library.

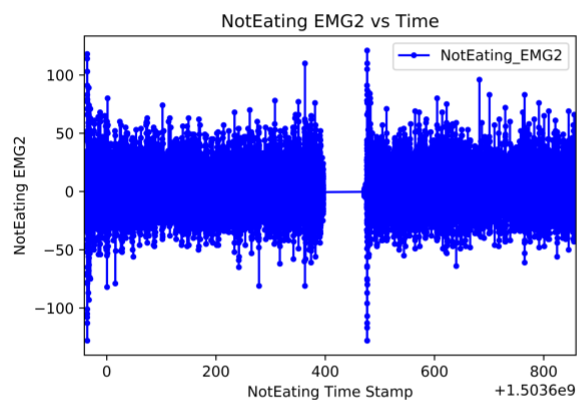
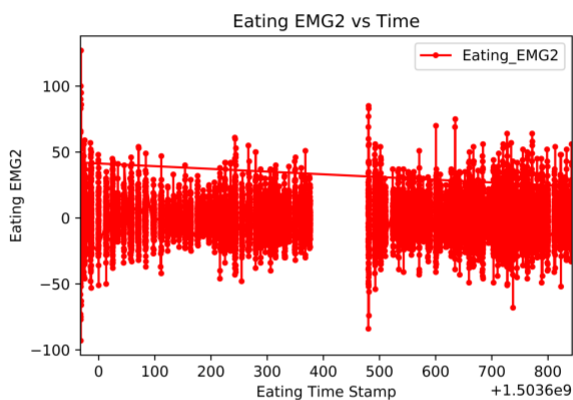
The python code is given in the file 'RawDataPlot.py' and the Path: /Phase2

All the plots are in the Path: /Phase2/RawData_Plots

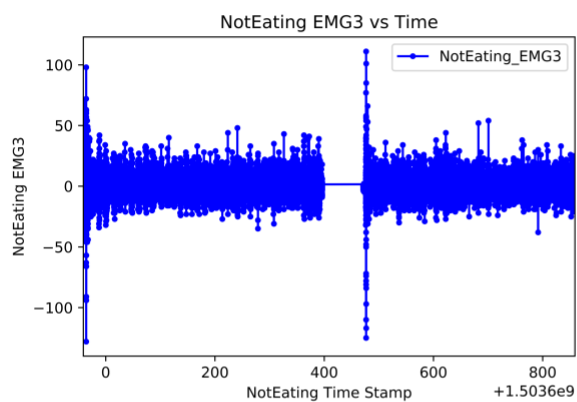
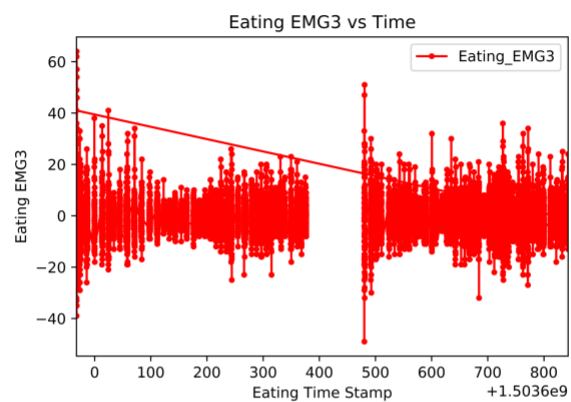
EMG 1:



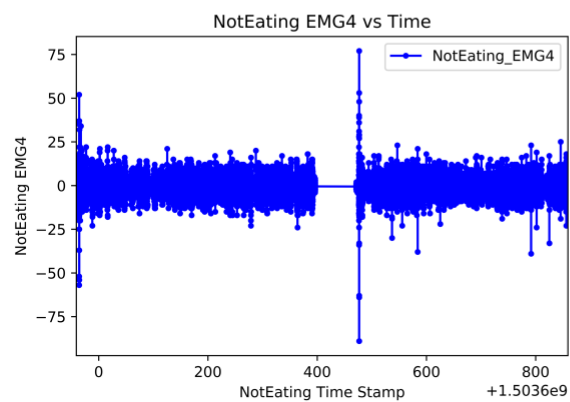
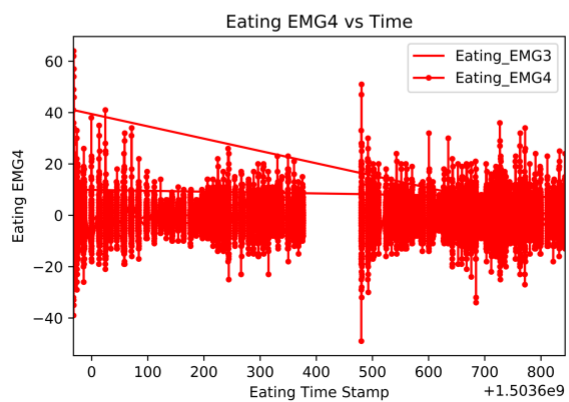
EMG 2:



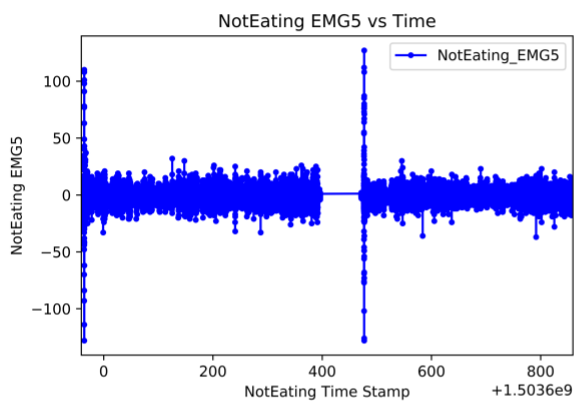
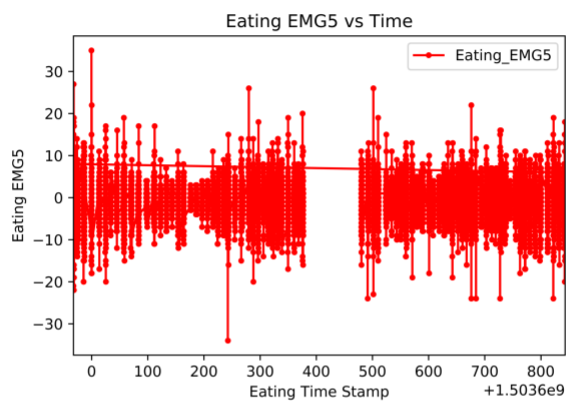
EMG 3:



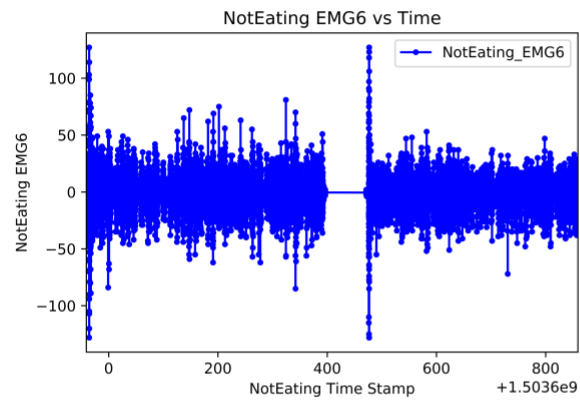
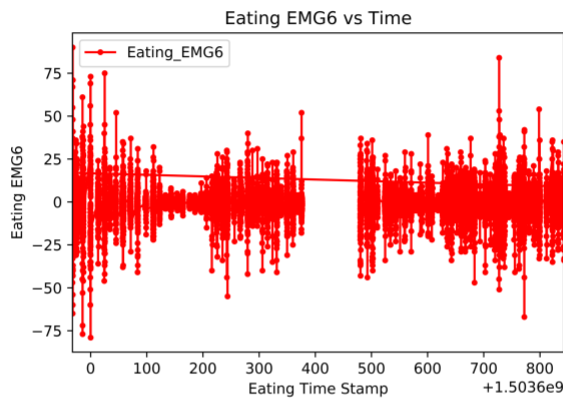
EMG 4:



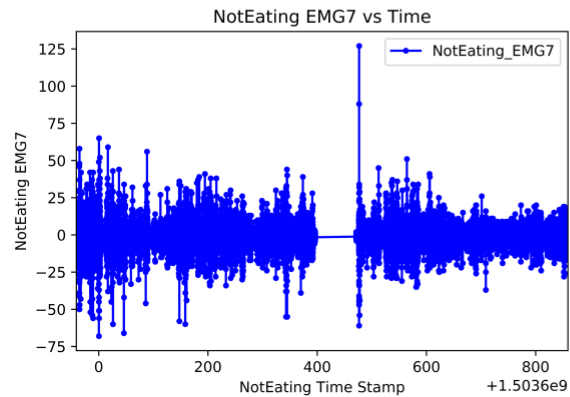
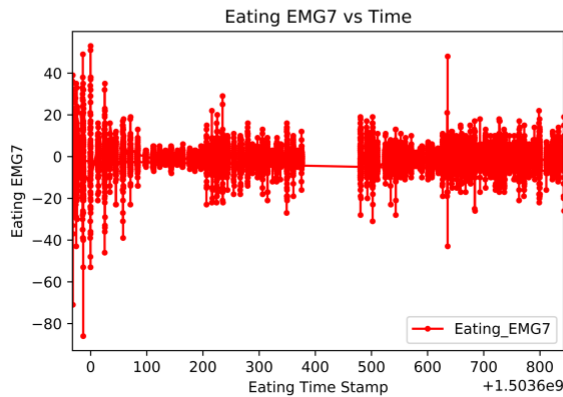
EMG 5:



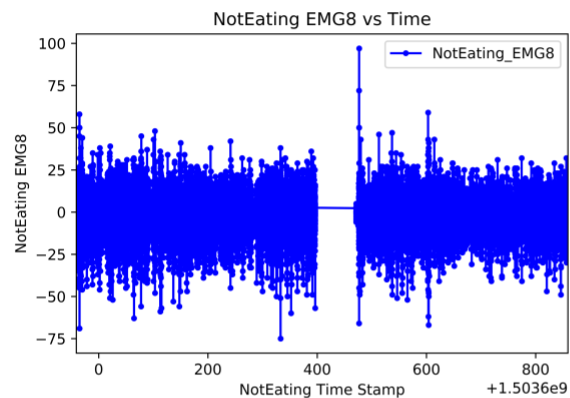
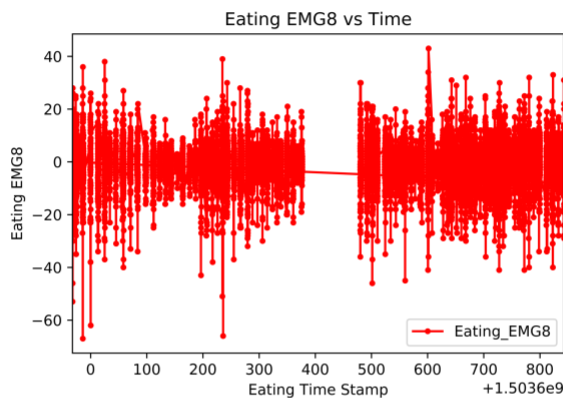
EMG 6:



EMG 7:



EMG 8:



Intuition behind choosing features:

For determining the eating actions from all the other actions, the EMG sensor data is vital. EMG Sensor or Electromyography sensor basically measures small electrical signals generated by muscles. This may include minute actions like lifting or moving arm, change in fist positions etc. Whenever there is a movement in the arm or fist, the mechanical change causes a change in electromechanical gradient which is detected by EMG sensor. From this we understand that variation of motion is directly related to the variation in the EMG sensor

data. Hence, based on this intuition Standard Deviation is the first feature that we selected. Also, we know that having both RMS along with Standard Deviation can provide flexibility if there is a non-linearity. So, RMS was selected as the second feature. Mean, Min and Max features are the most popularly used features especially when EMG sensor data is involved. So, based on this intuition we selected them. Now all the features selected so far are good indicators of the magnitude of the sensor data, but we also need an indicator for the periodicity of the data. We can use either the Fast Fourier Transform (FFT) or the Discrete Wavelet Transform (DWT) for measuring periodicity in the data. Many researchers and data scientists conducted a comparative study between the two, but there wasn't really a clear winner. The results varied based on the data that was used. Some of the researchers like Nihal Fatma Guler and Sabri Kocer [5] used the Fast Fourier Transform (FFT) for analyzing EMG signals and have got good results. Taking the work conducted by such researchers [1] [2] [3] [4] FFT was chosen as the last feature. Feature Extraction:

Mean:

Mean is the sum of all the sensor data (EMG1 to EMG 8) divided by the number of sensors, in our case it will be 8. Like this we calculate for every row. We get a new mean feature column. Standard Deviation:

The standard deviation is a statistic measure that measures the dispersion of a dataset relative to its mean and is calculated as the square root of the variance. The more data is spread out, higher will be the standard deviation. We calculated standard deviation for every row for EMG1 to EMG8. We get a new standard deviation feature column.

Min:

Min value of EMG1 to EMG8 for every row is selected. We get a new min feature column.

Max:

Max value of EMG1 to EMG8 for every row is selected. We get a new max feature column.

Root Mean Square:

In statistics, the root mean square is defined as the square root of the mean square. For every row, RMS is calculated for EMG1 to EMG8 sensor data. We get a new RMS feature column.

$$RMS = \sqrt{\frac{x_1^2 + x_2^2 + \dots + x_N^2}{N}}$$

Fast Fourier Transform (FFT):

We calculate the FFT values of EMG1 to EMG8 sensor data for every row using python's scikit-Learn library. We get 8 new FFT feature columns (FFT1 to FFT8). Now the problem is that the output of FFT consists of a real and an imaginary part. So, to make it easy for us, the absolute value of FFT was calculated by using the `numpy.abs()` method in python. The output of this gives 8 new absolute FFT columns (AMPFFT1 to AMPFFT8). All the code related to the Feature Extraction is in the file 'FeatureExtraction.py' Path: /Phase2

We combined the Eating Fork and Eating Spoon to one matrix called

'FinalEating.csv' Path: /Phase2

We also combined the Not Eating Fork and Not Eating Spoon to one matrix called

'FinalNotEating.csv' Path: /Phase2

After combining Eating Fork and Spoon samples there were 16180 samples out of which 8986 samples were from Fork and 7194 were from Spoon.

After combining NotEating Fork and Spoon samples there were 68513 samples out of which 30824 samples were from Fork and 37689 samples were from Spoon.

For plotting, we took equal number of samples for both Eating and Not Eating actions. Also, we took equal number of samples from Fork and Spoon in both the cases.

So, from Eating, 14000 samples were considered of which 7000 belonged to Fork and 7000 belonged to Spoon. From NotEating, 14000 samples were considered of which 7000 belonged to Fork and 7000 belonged to Spoon.

Plots for Eating Standard Deviation vs Time and NotEating Standard Deviation vs Time is in the following location:

Path: /Phase2/ Feature1_StandardDeviation_Plots

Plots for Eating Mean vs Time and NotEating Mean vs Time is in the following location:

Path: /Phase2/ Feature2_Mean_Plots

Plots for Eating Min vs Time and NotEating Min vs Time is in the following location:

Path: /Phase2/ Feature3_Min _Plots

Plots for Eating Max vs Time and NotEating Max vs Time is in the following location:

Path: /Phase2/ Feature4_Max _Plots

Plots for Eating RMS vs Time and NotEating RMS vs Time is in the following location:

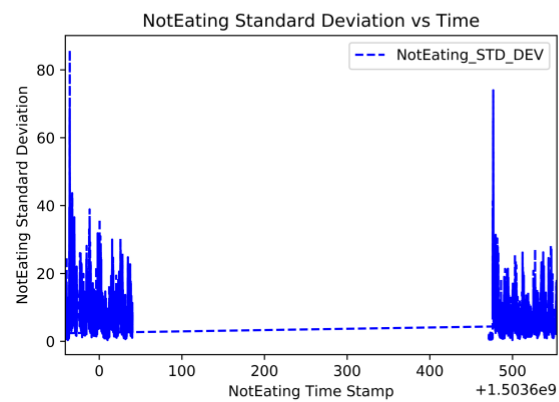
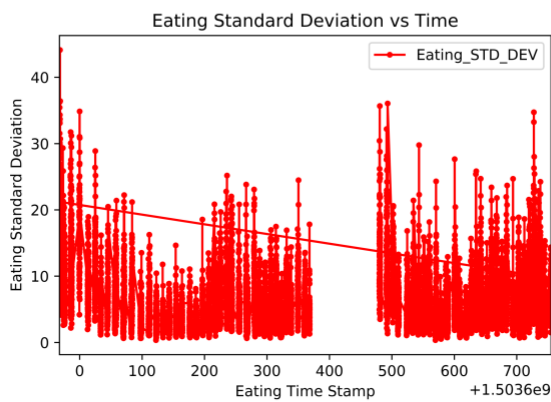
Path: /Phase2/ Feature5_RMS_Plots

Plots for Eating AMPFFT1 to AMPFFT8 vs Time and NotEating AMPFFT1 to AMPFFT8 vs Time is in the following location:

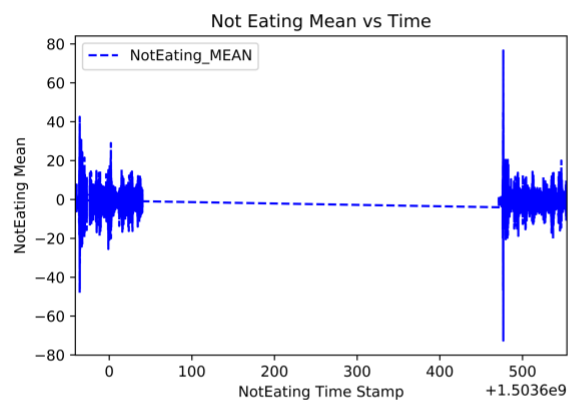
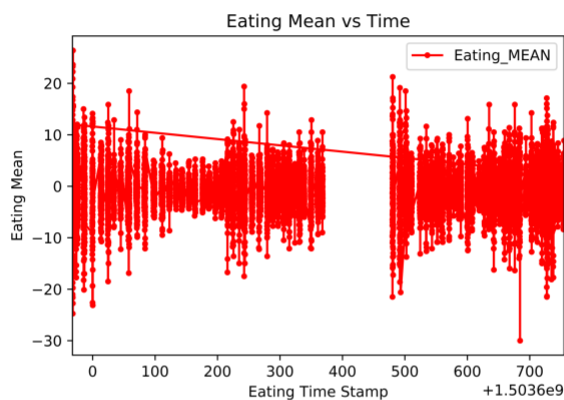
Path: /Phase2/ Feature6_FFT_Plots

Python code for Plots is in file 'FeatureExtractionPlot.py' in Path: /Phase2

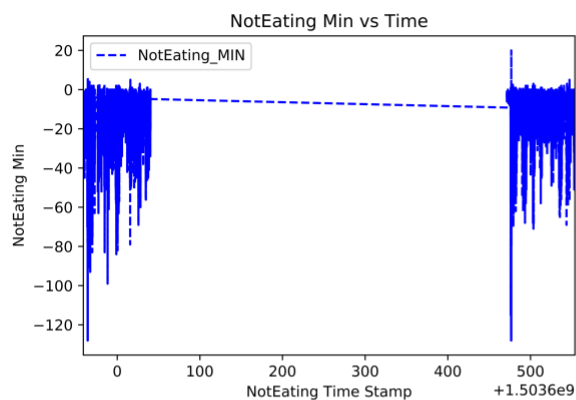
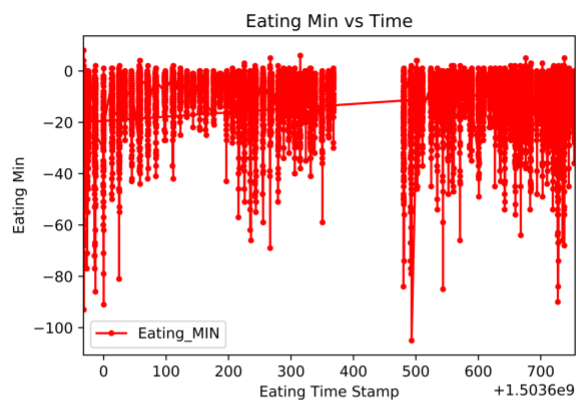
Standard Deviation:



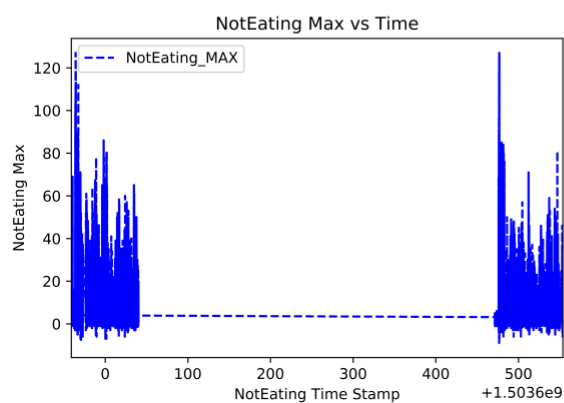
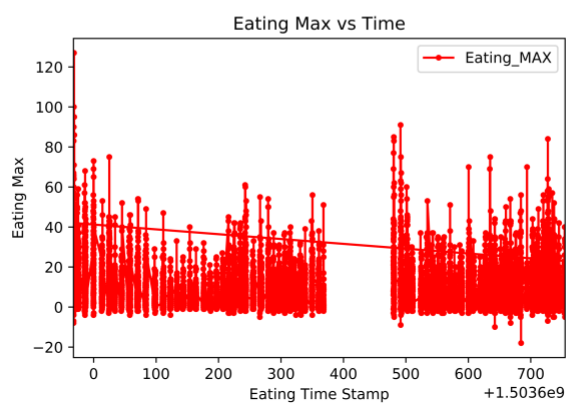
Mean:



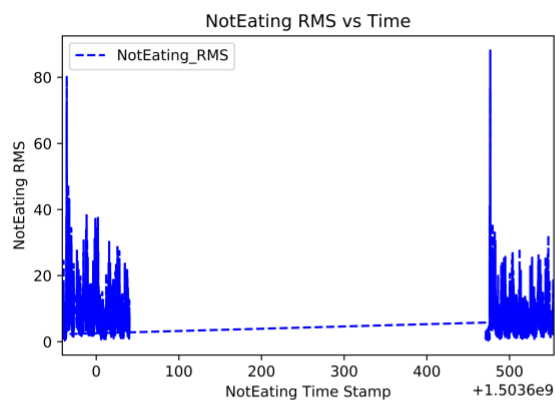
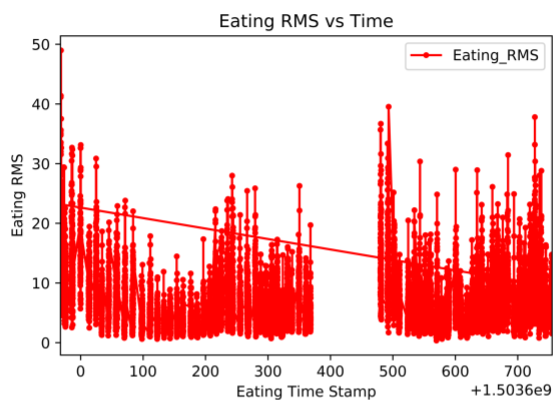
Min:



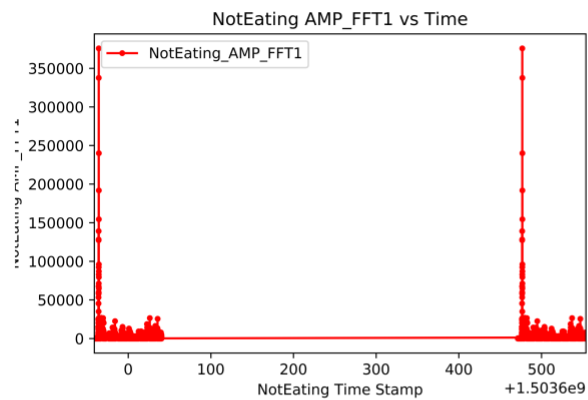
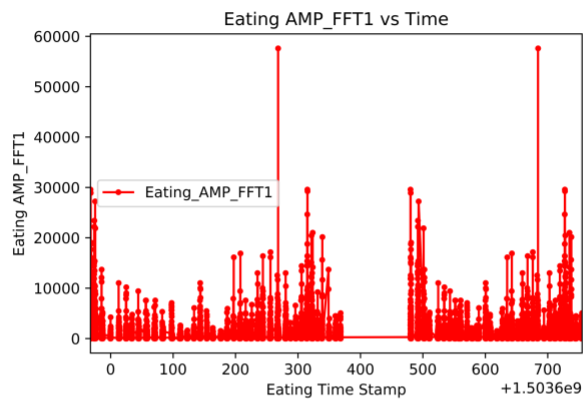
Max:



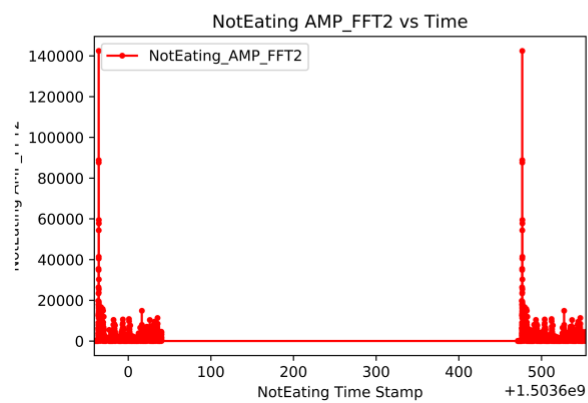
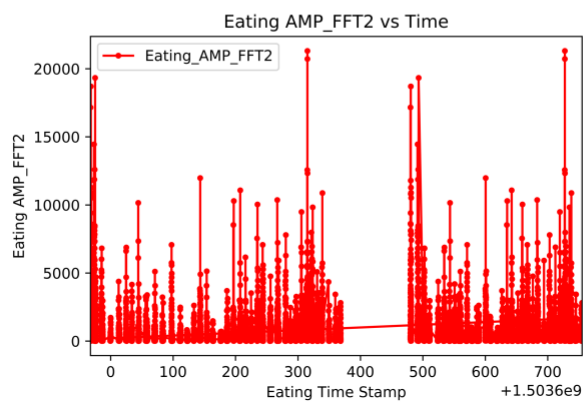
RMS:



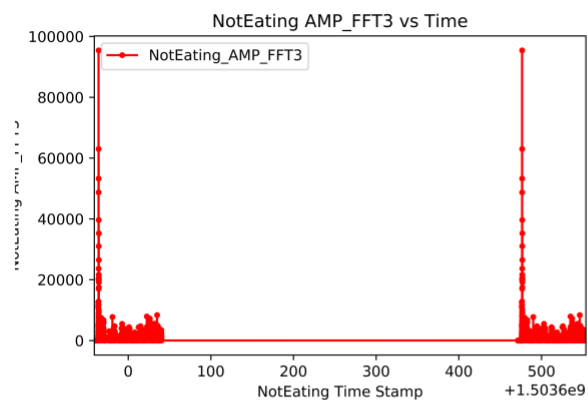
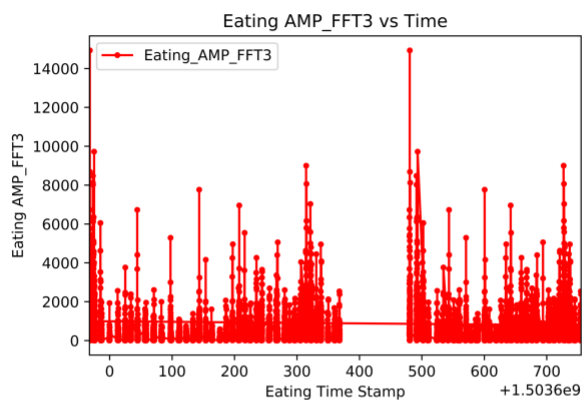
AMPFFT1:



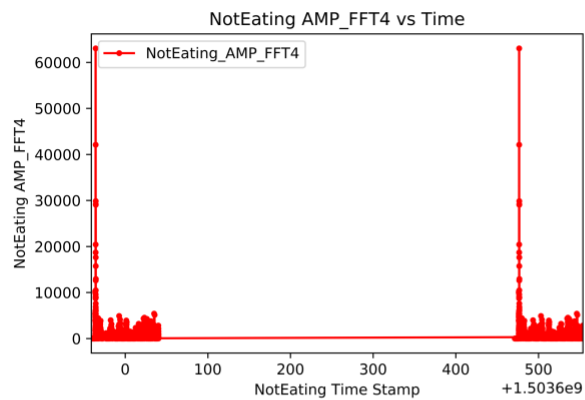
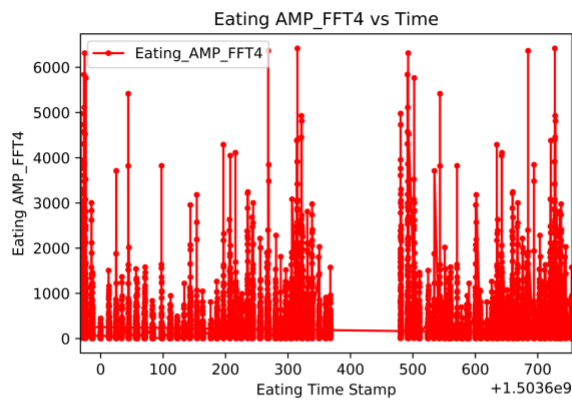
AMPFFT2:



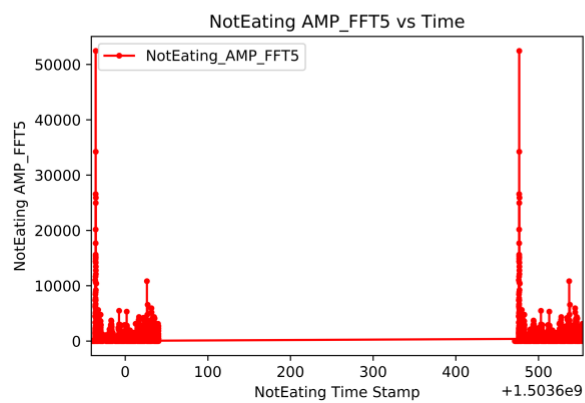
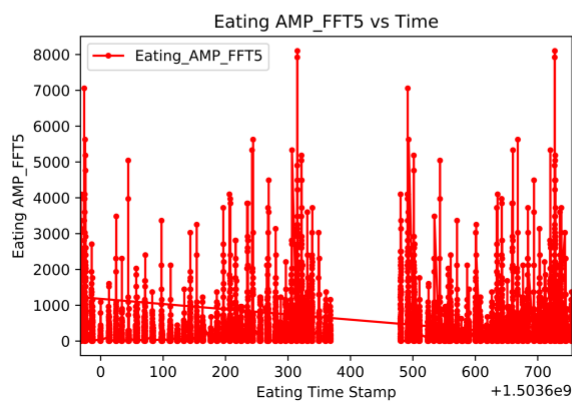
AMPFFT3:



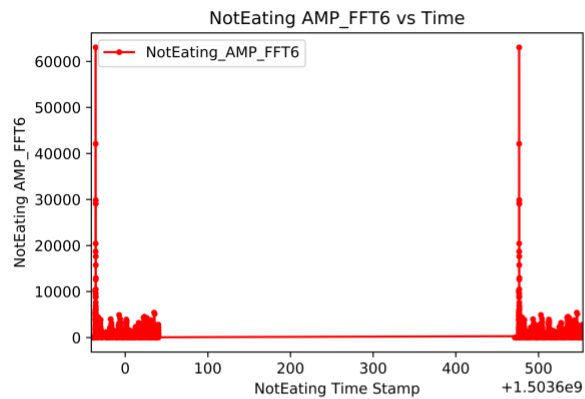
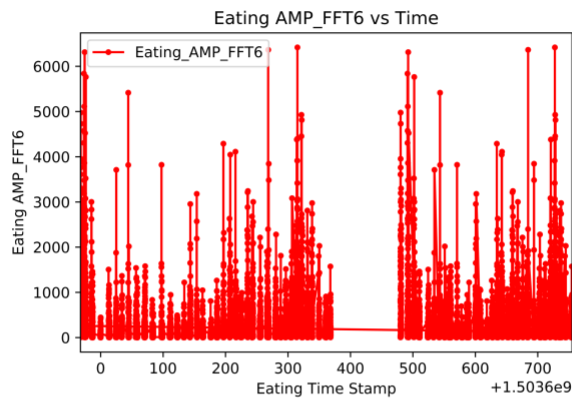
AMPFFT4:



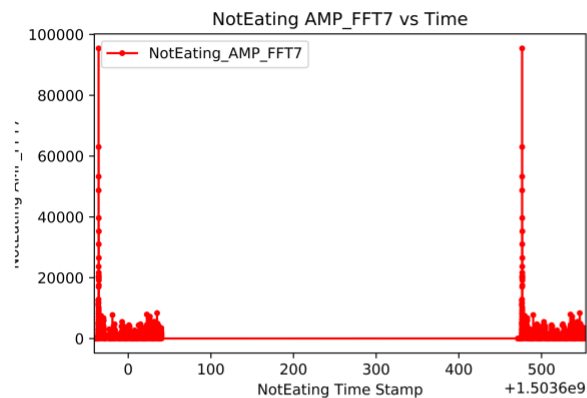
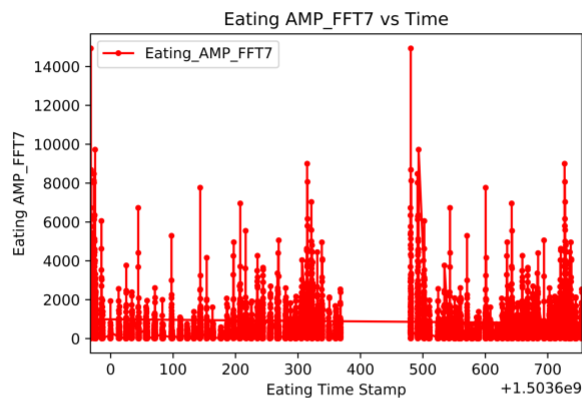
AMPFFT5:



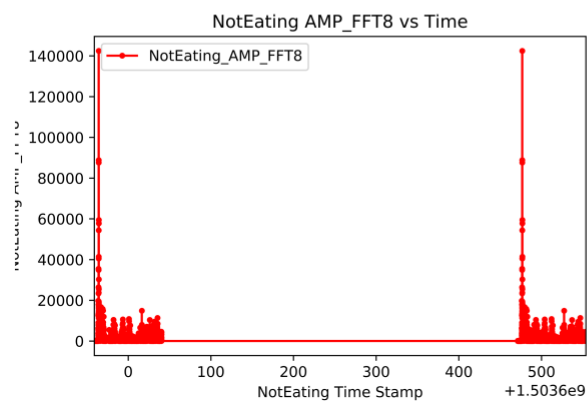
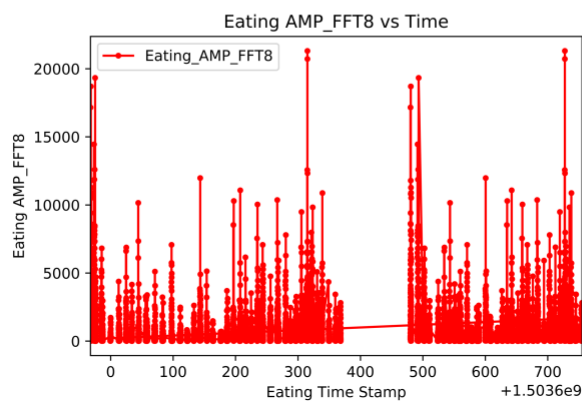
AMPFFT6:



AMPFFT7:



AMPFFT8:



From the plots above, we can see that there is a considerable difference between Standard Deviation, Mean, RMS and some FFT values of Eating and Not Eating actions. So, our initial intuitions about the features selected holds true.

Project Phase III: Feature Selection

At the end of Phase 2, we selected certain features based on intuition for both Eating and Not Eating actions. We got two matrices 'FinalEating.csv' and 'FinalNotEating.csv'. But Principal Component Analysis (PCA) takes only one matrix as input. So, we need to combine the two matrices into one feature matrix. Using python, we appended all the samples from FinalEating and FinalNotEating matrices to 'FeatureMatrix' matrix. Now we have a total of 84693 samples in this Feature matrix of which 16180 belonged to Eating and 68513 belonged to Not Eating. Also, looking at all the features, we know that we do not need FFT1 to FFT8 features as we can use absolute values of them AMPFFT1 to AMPFFT8. Hence, we dropped those features from the feature matrix. We also dropped the SampleNumber, TimeStamp and EMG1 to EMG8 columns as we do not need them for PCA. The feature matrix after this will be 84693x13 where there are Standard Deviation, Mean, Min, Max, RMS and 8 FFT features (AMPFFT1 to AMPFFT8).

The python code is in the file 'PCA.py' in the Path: /Phase3

The feature matrix is in the file 'FeatureMatrix.csv' in the Path: /Phase3

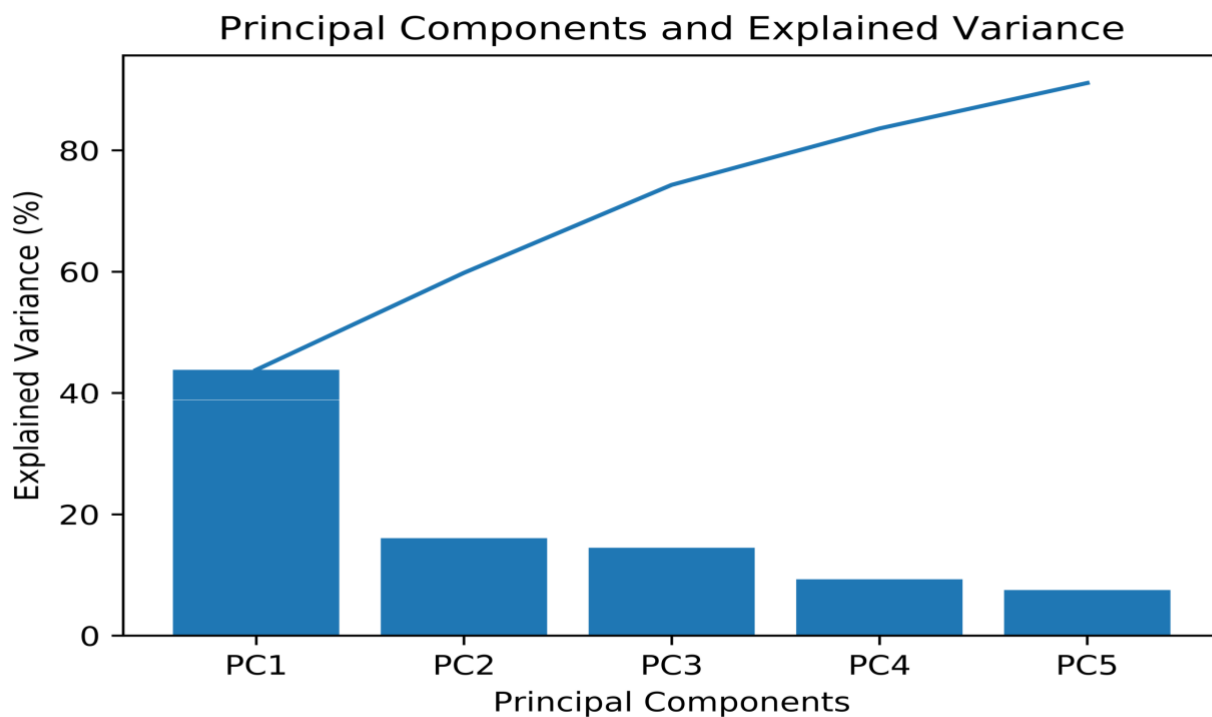
Principal Component Analysis (PCA):

Python's scikit-Learn library has an out of the box PCA method that has been used in this project.

StandardScaler method has also been used to normalize the data in the feature matrix. The python code for this is in the file 'PCA.py' in the Path: /Phase3

After the execution of PCA we got the new matrix 'NewFeatureMatrix.csv' with the principal components. The new feature matrix is in the file 'NewFeatureMatrix.csv' in Path: /Phase3

So why Eigen vectors turned out the way they did? For doing Principal Component Analysis (PCA), we first need to calculate the covariance matrix for the all the features selected. The covariance matrix defines two things: spread also called variance and orientation also called covariance. After this, we get the eigen vectors that tell us about the direction in which there is the highest spread or variance [6]. Eigen values are basically the magnitudes of these vectors. These eigen values in decreasing order of variance become the new principal components. Ideally, after performing PCA we should be seeing the explained variance ratio of the first two principal components to be the maximum. This means that a vast majority of variance in the data can be captured by the first few principal components thus reducing the number of dimensions which is our final objective of this exercise. If we look at the new feature matrix that is obtained above, we can see the eigen values following the exact same pattern as the ideal behavior. Thus, we can conclude that Eigen vectors and Eigen values turned out as expected.



The explained variance ratio of the first 5 principal components is given below:

[0.4380011 0.16043249 0.14470935 0.09295237 0.0751009]

This implied to us that about 91% of variance in the data is captured in the first 5 components.

The explained variance ratio of the first 6 principal components is given below:

[0.4380011 0.16043249 0.14470935 0.09295237 0.0751009 0.04236383] About

95% of the variance in the data is captured in the first 10 components.

Assuming capturing 91% of variance in the data is good enough for us, PCA has effectively reduced the feature matrix from 84693x13 (13 features) to 84693x5 (5 features). Hence, we can say that doing PCA was very helpful in reducing the dimensions of the dataset.

Bibliography

1. ABDELKAREEM BEDRI, RICHARD LI, MALCOLM HAYNES, RAJ PRATEEK KOSARAJU, ISHAAN GROVER, TEMILOLUWA PRIOLEAU, MIN YAN BEH, MAYANK GOEL, THAD STARNER, and GREGORY ABOWD. "EarBit: Using Wearable Sensors to Detect Eating Episodes in Unconstrained Environments." n.d.
2. Ben Nicholls, Chee Siang Ang, Kanjo Eiman, Panote Siriaraya, Woon-Hong Yeo. "An EMG-based Eating Behaviour Monitoring System with Haptic Feedback to Promote Mindful Eating." n.d.
3. Chee Siang Ang, Eiman Kanjo, Athanasios Tsanas. "An EMG-based Eating Behaviour Monitoring System with Haptic Feedback to Promote Mindful Eating." n.d.
4. Jingwei Too, Abdul Rahim Abdullah, Norhashimah Mohd Saad, Weihown Tee. "EMG Feature Selection and Classification Using a Pbest-Guide Binary Particle Swarm Optimization." *Computation*, 2019.
5. Nihal Fatma Guler, Sabri Kocer. "Classification of EMG Signals Using PCA and FFT." *Journal of Medical Systems*, 2005.
6. Alto, Valentino. *Towards Data Science*. July 13, 2019. <https://towardsdatascience.com/pca-eigenvectors-and-eigenvalues-1f968bc6777a>.